武漢大學
WUHAN UNIVERSITY

《分布式软件原理与技术》
第五讲 分布式事务

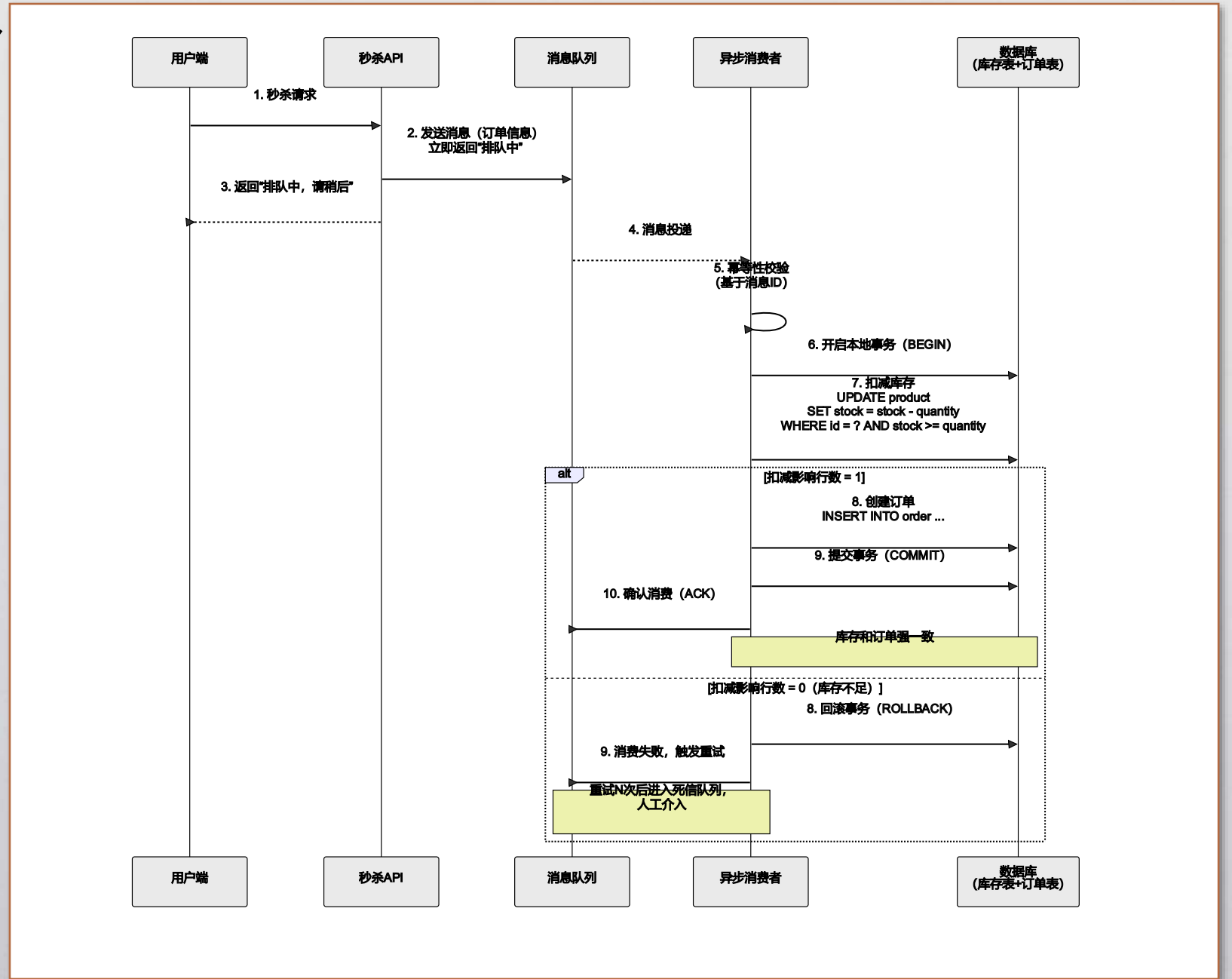
计算机学院 贾向阳

武漢大學
WUHAN UNIVERSITY

1、 分布式事务简介

单体版的秒杀

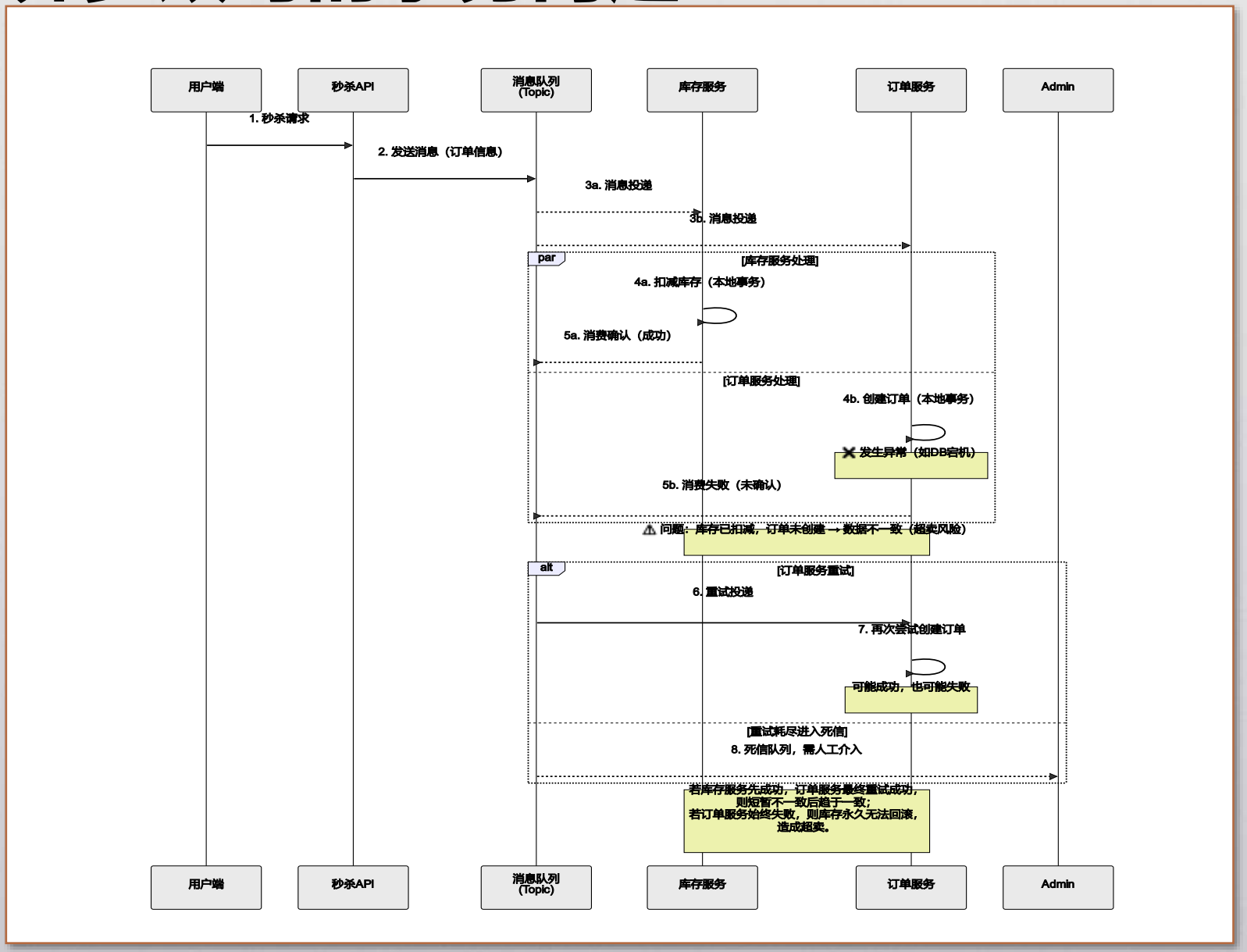
- 场景：订单表和库存表在同一个数据库中
- 事务：订单服务可以在一个本地事务中执行创建订单和扣减库存操作。任何一个操作失败，其他都要回滚。
- **订单和库存是强一致的。**



微服务版秒杀：异步双写的事务问题

- **场景：**订单服务库存服务分别是两个微服务，有各自数据库。
- **异步双写：**订单服务和库存服务分别监听消息队列，然后进行订单创建与库存扣减
- **事务问题：没有事务保障，**如果任何一个服务失败，会出现**不一致问题**，如发生超卖或少卖。

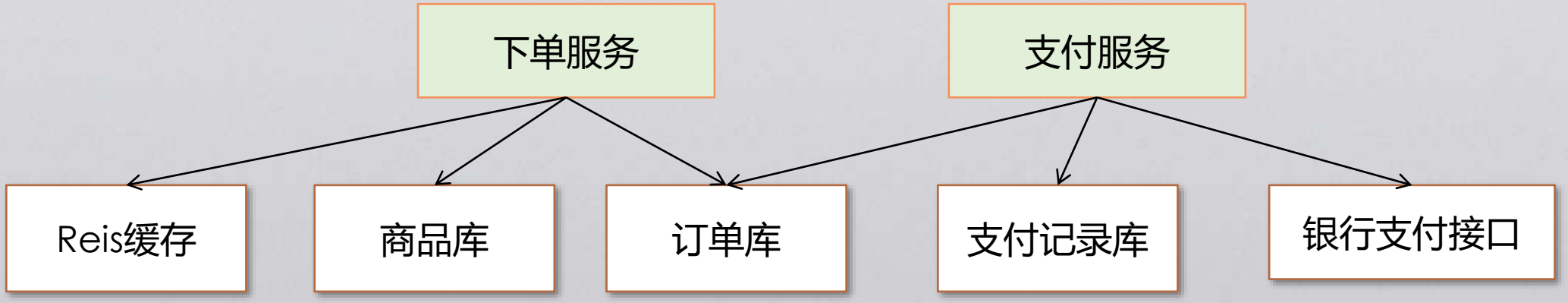
- 缺一个协调者，来让已完成的服务回滚数据。



从单体到分布式：事务的“进化困境”

从单体到分布式

- 回顾：单机数据库中，事务通过本地ACID机制轻松实现。
- 现状：分布式软件中，事务涉及到微服务、分库分表、异构存储（MySQL + Redis + ES）
- 问题：
  - 一个业务操作（如下单）可能涉及多个数据库，如何保障事务？
  - 如果涉及到外部服务，如银行支付接口，如何保证和数据库的一致性？



分布式环境下的事务挑战

武漢大學
WUHAN UNIVERSITY

两种哲学：刚性 vs 柔性

- **刚性事务（强一致性）**
  - 追求与传统ACID相同的效果，强一致、原子性。
  - 典型模式：2PC（基于XA协议）、3PC，以及现代数据库（如OceanBase、TiDB）的全局一致性读
  - 代价：性能较低、锁范围大、可用性有损失。
- **柔性事务（最终一致性）**
  - 接受短时间内数据不一致，但保证最终数据一致。
  - 典型模式：TCC、SAGA、本地消息表、事务消息
  - 优势：高可用、高性能、适合微服务架构

没有银弹，只有合适的场景

两个极端案例对比

- **案例A：银行转账扣款**
  - 要求：绝对不允许出现A扣了钱B没收到，或者重复扣款。
  - 倾向：强一致（刚性事务），宁可短暂不可用也不能错账。
- **案例B：电商下单发优惠券**
  - 要求：下单成功是核心，优惠券晚几分钟发也可以接受；如果发券失败，后台重试即可。**只要数据没丢，最终会重试成功。**
  - 倾向：最终一致（柔性事务），优先保证下单成功和系统可用性。

- **结论：技术选型的起点是业务对一致性、可用性、性能的优先级排序**

2、 经典分布式事务

分布式事务的核心挑战

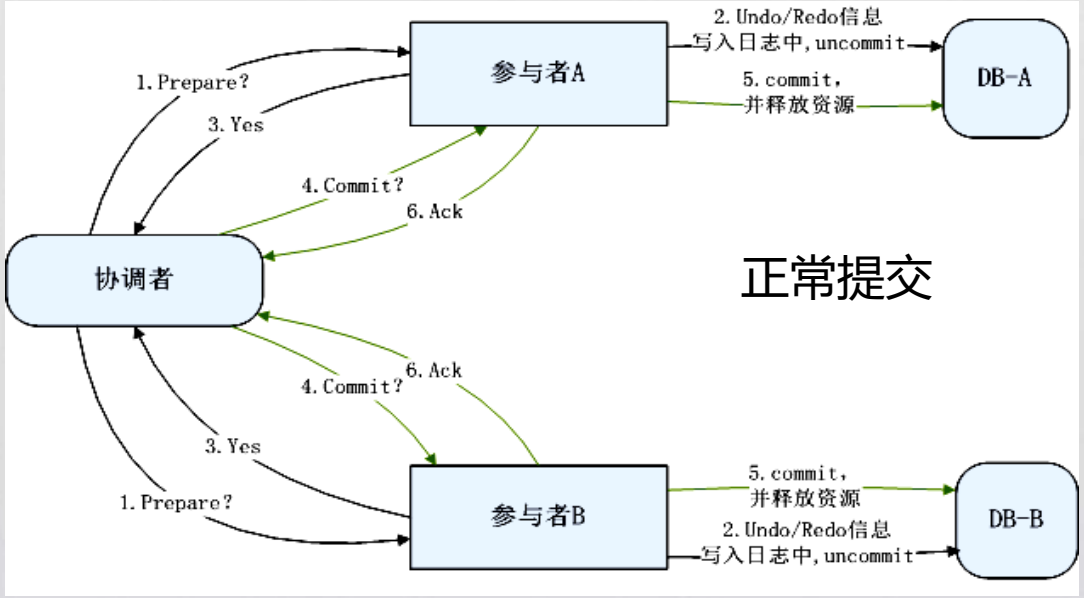
- **原子性问题**
  - 如何让多个节点的操作像一个整体一样提交或回滚?
- **隔离性问题**
  - 分布式环境下，如何控制并发事务之间的可见性?
- **容错问题**
  - 网络分区无法通信了怎么办？协调者挂了怎么办？参与者挂了怎么办?
- **性能问题**
  - 如何在保证一致性的前提下，减少锁持有时间和网络交互次数?

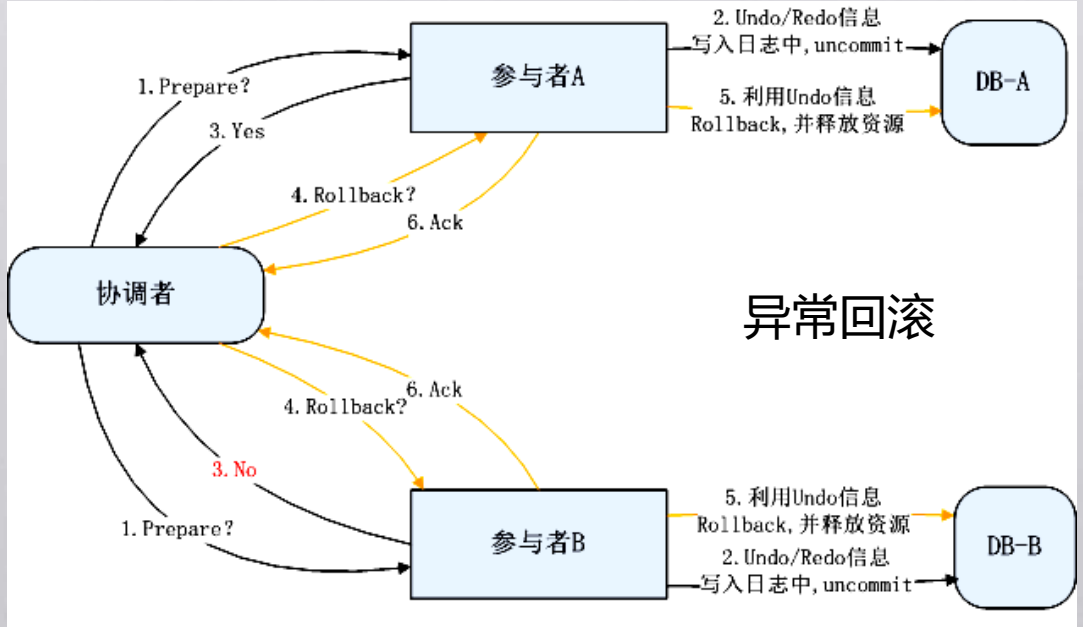
XA规范与2PC协议

- **XA规范（X/Open Distributed Transaction Processing）**
  - XA是一个规范/标准，由 X/Open 组织定义的分布式事务处理规范
  - 定义了事务管理器（TM） 与资源管理器（RM，如数据库、消息队列等） 之间的交互接口
- **2PC协议（Two-Phase Commit）**
  - 2PC是XA 规范指定的原子提交协议，
  - 定义了TM与RM的交互算法，保证事务在所有参与者之间要么全部提交，要么全部回滚。

2PC协议

- **阶段一：准备/投票阶段**
  - 协调者向所有参与者发送“Prepare”请求。
  - 每个参与者执行事务（写日志），但不提交，返回Yes或No。
- **阶段二：提交/回滚阶段**
  - 协调者收集所有投票：如果全是Yes，发送Commit；如果有No或超时，发送Rollback。
  - 每个参与者收到指令后，执行Commit或Rollback



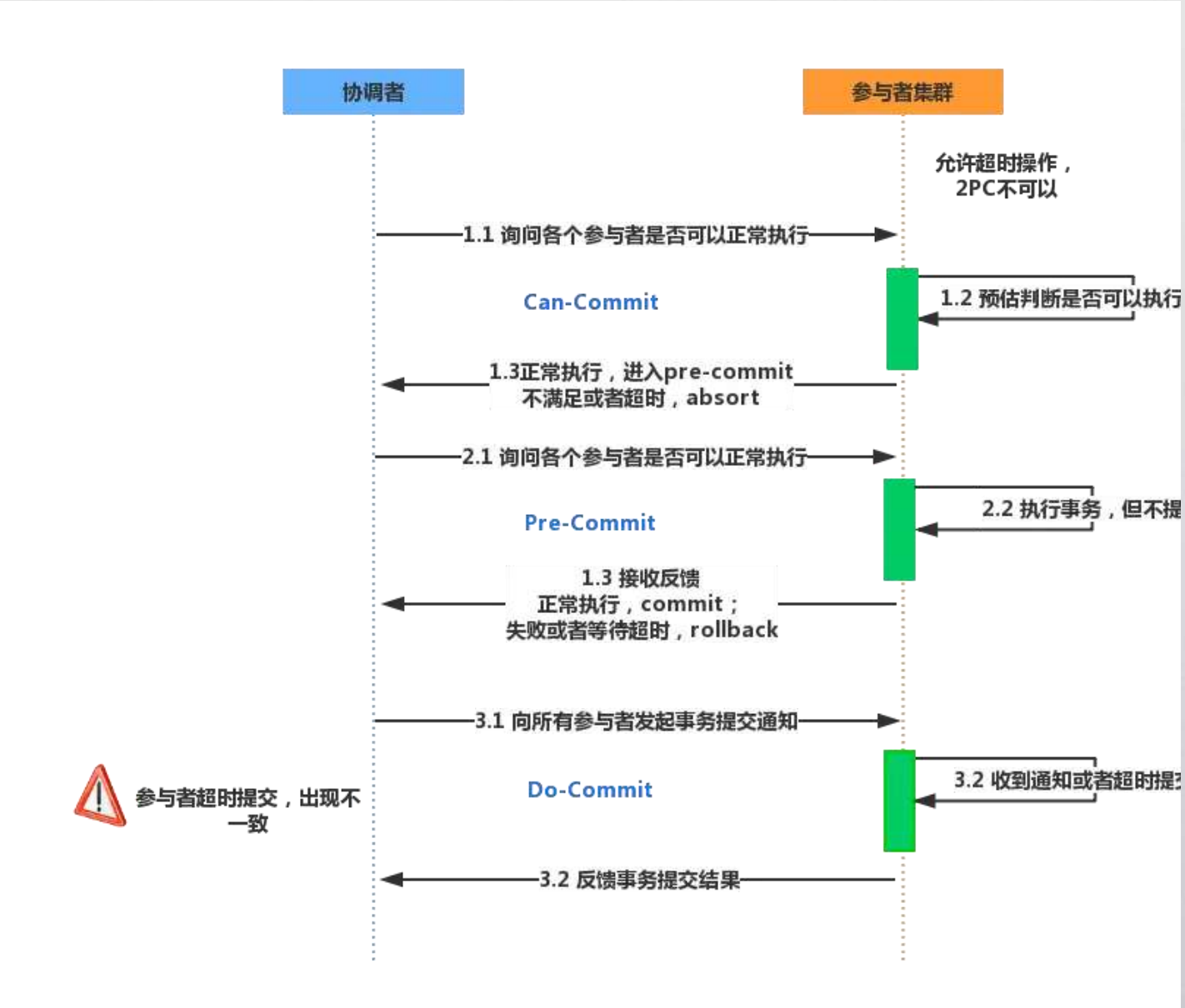


2PC协议的缺陷

- **主要痛点**
  - **单点问题：**如果协调者出现异常和故障，那么整个流程将无法运行。
  - **同步阻塞**：各个参与者需要等待其他参与者响应，协调者需要等待所有参与者回复，才能进行下一步的动作。
  - **数据不一致**：第二阶段中，可能只有部分参与者收到commit/rollback命令，而另一部分没有收到，导致数据不一致。

3PC协议

- **阶段一：CanCommit**
  - 协调者询问
  - 参与者反馈是否可执行事务（仅做健康检查）
- **阶段二：PreCommit**
  - 协调者通知
  - 参与者执行事务（写日志，但不提交），然后返回结果。
- **阶段三：DoCommit**
  - 协调者发送最终提交指令。
  - 参与者完成事务提交



3PC协议相对2PC的改进

特性	2PC	3PC
可用性	参与者执行操作后，才发现有节点不可用	先确认可用性，再执行操作。
阻塞	参与者可能无限等待协调者	参与者超时后自动提交，减少阻塞
一致性风险	协调者故障可能导致部分参与者提交、部分回滚	参与者自主提交，仍可能在网络分区下出现不一致，但概率更低

3PC协议的局限

局限

- **不能完全解决一致性问题**：如果网络分区导致部分参与者收到PreCommit而另一部分未收到，可能造成“部分提交、部分回滚”的不一致情况。但它显著缓解了2PC中协调者故障导致的“阻塞+死等”问题。
- **复杂度增加**：多一次网络交互，性能反而可能下降。

结论

- 2PC 和 3PC 主要解决原子性，但没有完美解决一致性问题，且性能较低。
- 实际工程中，如果追求强一致性，Paxos或Raft等主从复制协议通常是比3PC更优的选择。

现代分布式数据库的事务方案

- **现代数据库的事务方案**
  - PolarDB-X、OceanBase这类现代分布式数据库，采用2PC协议，但通过Paxos或Raft一致性算法，以及分层架构和组合设计大幅缓解或规避其问题
- **高可用协调者集群**
  - 协调者从单点变为高可用集群，任一节点故障不影响整体事务流程
  - 在PolarDB-X中，协调者通过负载均衡多活部署，任意计算节点宕机后客户端自动切换。进行中的事务状态会记录在持久化存储（如GMS）中，由其他计算节点接管。
  - 在OceanBase中，多个节点通过Paxos选举领导者进行事务协调，领导者故障后自动选举新领导者，事务状态通过Paxos日志同步，可恢复。

现代分布式数据库的事务方案

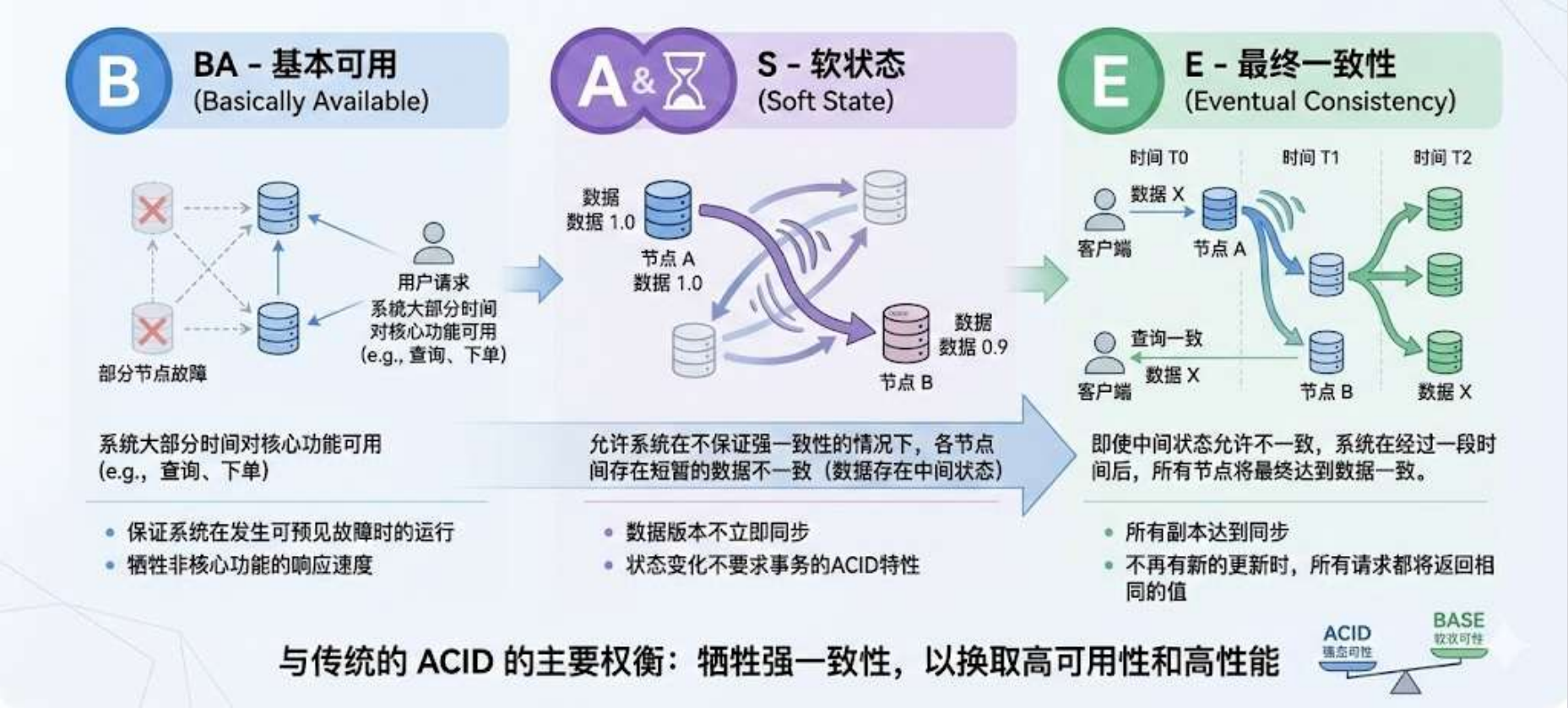
- **同步阻塞问题**
  - 2PC协议参与者从Prepare到Commit/Rollback期间，一直持有本地锁资源，等待协调者指令。协调者也要等待所有参与者回复后才能进行下一步
- **解决方案**
  - MVCC（多版本并发控制）
    - MVCC为每行数据维护多个历史版本，每个版本关联一个事务ID（或时间戳）。事务读取数据时，只读取在时间戳之前已提交的数据版本，而无需等待正在修改该数据的事务 MVCC 使得读操作不阻塞写操作，写操作也不阻塞读操作。2PC协议的事务在Prepare阶段虽然加锁，但其他事务可以读取该行的历史版本，阻塞范围被压缩到只针对写-写冲突。
  - 全局一致快照
    - 通过中心时钟服务分配全局递增的时间戳，所有节点使用同一时间源。
    - 事务开始时获取快照时间戳，读取时通过RPC获取其他节点数据版本，在本地判断可见性

一致性问题

- **一致性问题**
  - 协调者发送Commit后宕机，部分参与者收到Commit，另一部分未收到，导致数据不一致。
- **解决方案**
  - **决策日志持久化：**协调者在发送Commit之前，先将“全局事务已提交”这一决策通过Paxos/Raft同步到多数派协调者节点。
  - **参与者的幂等处理：**参与者接口设计为幂等，重复收到Commit不会造成错误。
  - **多副本数据分片**：存储层的每个数据分片本身也通过Paxos/Raft实现多副本，确保一旦提交，数据不会因节点故障而丢失。
  - **重试与恢复：**即使协调者在发送Commit过程中宕机，新选出的协调者可以读取持久化的决策日志，向未收到Commit的参与者重发提交指令。

3、 现代分布式事务协议

柔性事务的理论基础-BASE理论



- 允许非核心业务出错或降级：如优惠券服务超时，允许跳过。
- 需要处理软状态：如先“锁定库存”，订单创建成功或失败后再更新库存。
- 需要保障最终一致性：例如通过“确认/取消”或重试来保障数据一致性

从数据库层到业务层事务

- 为什么需要业务层事务?
  - 数据库层的XA协议（2PC）锁资源时间长，对数据库压力大。
  - 在微服务架构中，事务往往跨越多个服务，涉及MySQL、Redis、ES等异构存储，无法使用数据库层的2PC协议。
- 趋势
  - 将事务控制权上浮到业务层，通过业务逻辑来保证最终一致性。
- 主流方案
  - TCC、SAGA、基于消息的一致性。

TCC 模式——经典柔性事务

- **TCC (Try-Confirm-Cancel)**
  - 一种侵入性较强的业务层事务方案
  - **业务层需要实现三个接口：**
    - Try：预留资源（冻结资金、锁定库存）
    - Confirm：确认执行（扣减资源，提交）
    - Cancel：取消执行（释放预留资源）
  - **事务流程：**主业务服务调用所有子服务的Try接口。所有Try成功，调用Confirm；任一Try失败，调用Cancel。
  - **优点：**性能高（无锁库表，锁业务资源），最终一致性。
  - **缺点：**对业务代码侵入大，开发成本高

TCC 示例

正常流程（全部成功）

1. 事务发起者（TM）调用所有参与者的 Try 接口。
   ├─ 参与者 A：冻结资金 100 元。
   ├─ 参与者 B：锁定库存 1 件。
   └─ 参与者 C：预占积分 10 分。
2. 所有参与者返回 Try 成功。
3. TM 调用所有参与者的 Confirm 接口。
   ├─ 参与者 A：扣减资金，解冻无效。
   ├─ 参与者 B：扣除库存，释放锁定。
   └─ 参与者 C：增加积分，释放预占。
4. 事务结束。

异常流程（某个参与者 Try 失败或超时）

1. TM 调用参与者 A Try 成功。
2. TM 调用参与者 B Try 失败（如库存不足）。
3. TM 不再调用后续参与者 C 的 Try。
4. TM 调用所有已成功 Try 的参与者的 Cancel 接口。
   ├─ 参与者 A：解冻资金。
   └─ 参与者 B：无需 Cancel（因为 Try 未成功）。
5. 事务结束，整体已回滚。

Confirm 或 Cancel 接口可能因网络、节点故障等原因执行失败。
TM 会不断重试失败的 Confirm 或 Cancel，直到成功。
要求：**Confirm 和 Cancel 接口必须设计为幂等，多次调用结果相同。**

TCC 应用场景

- **电商下单**
  - Try 阶段锁定库存和优惠券
  - Confirm 阶段扣减库存、扣减优惠券
  - Cancel 阶段释放库存和优惠券的锁定。
- **支付交易**
  - Try 阶段冻结资金，Confirm 阶段扣款，Cancel 阶段解冻。
- **积分兑换**
  - Try 阶段预占积分，Confirm 阶段扣减积分并发放奖品，Cancel 阶段释放积分。
- **账户转账**
  - Try 阶段冻结转出账户资金
  - Confirm 阶段扣减转出账户、增加转入账户
  - Cancel 阶段解冻转出账户。

SAGA 模式 ——补偿式事务

SAGA事务

- SAGA 起源于 1987 年的论文，适用于长事务
- **核心思想：**将一个长事务拆分为多个本地短事务（T1, T2, ..., Tn）。每个短事务 Ti 对应一个补偿操作 Ci。
- **执行策略**：
  - **正常提交**：T1, T2, T3 ... 全部成功 -> 结束。
  - **补偿回滚**：如果 Ti 失败，执行 C(i-1) ... C1，补偿之前已成功的操作。
- **与TCC区别：**SAGA没有“Try”阶段，直接执行提交，出错了再反向补偿。

Seata分布式事务框架

- **Seata框架**
  - 阿里开源的，目前最流行的分布式事务框架
- **支持多种事务模式**
  - **AT模式**（Automatic Transaction）
    - 无侵入，基于数据源代理（类似2PC，但锁粒度优化）。
    - 解析SQL，记录“undo_log”（镜像数据）。
    - 默认全局锁 + 本地事务，性能优于传统2PC。
  - **TCC模式**：业务侵入，高性能。
  - **SAGA模式**：长事务，状态机驱动。

基于Seata实现TCC事务

（1）参与者需要实现三个接口

```
@LocalTCC
public interface ProductTccService {

    @TwoPhaseBusinessAction(name = "productTcc", commitMethod = "confirm", rollbackMethod = "cancel")
    boolean tryDeductStock(
            @BusinessActionContextParameter(paramName = "productId") Long productId,
            @BusinessActionContextParameter(paramName = "quantity") Integer quantity,
            BusinessActionCont

    boolean confirm(BusinessAc

    boolean cancel(BusinessAct
}
```

```
@LocalTCC
public interface OrderTccService {

    @TwoPhaseBusinessAction(name = "orderTcc", commitMethod = "confirm", rollbackMethod = "cancel")
    boolean tryCreateOrder(
            @BusinessActionContextParameter(paramName = "userId") Long userId,
            @BusinessActionContextParameter(paramName = "productId") Long productId,
            @BusinessActionContextParameter(paramName = "quantity") Integer quantity,
            BusinessActionContext context);

    boolean confirm(BusinessActionContext context);

    boolean cancel(BusinessActionContext context);
}
```

基于Seata实现TCC事务

（2）商品服务实现三个接口

```java
@Service
public class ProductTccServiceImpl implements ProductTccService {

    @Autowired
    private ProductMapper productMapper;

    @Override
    @Transactional
    public boolean tryDeductStock(Long productId, Integer quantity, BusinessActionContext context) {
        log.info("Product Try: 冻结库存, productId={}, quantity={}", productId, quantity);
        int rows = productMapper.freezeStock(productId, quantity);
        if (rows == 1) {
            context.getActionContext().put("productId", productId);
            context.getActionContext().put("quantity", quantity);
            return true;
        } else {
            log.warn("库存不足或并发冻结失败");
            return false;
        }
    }
```

武漢大學 WUHAN UNIVERSITY

基于Seata实现TCC事务

（2）商品服务实现三个接口

```java
@Override
@Transactional
public boolean confirm(BusinessActionContext
    Map<String, Object> params = context.get
    Long productId = (Long) params.get("pro
    Integer quantity = (Integer) params.get
    log.info("Product Confirm: 真正扣减库存,
    int rows = productMapper.confirmStock(p
    return rows == 1;
}

@Override
@Transactional
public boolean cancel(BusinessActionContext
    Map<String, Object> params = context.getActionContext();
    Long productId = (Long) params.get("productId");
    Integer quantity = (Integer) params.get("quantity");
    log.info("Product Cancel: 释放冻结库存, productId={}, quantity={}", productId, quantity);
    int rows = productMapper.cancelStock(productId, quantity);
    // 空回滚: 即使 rows == 0 也返回 true (表示已处理)
    return true;
}
```

```java
@Mapper
public interface ProductMapper {
    @Update("UPDATE product SET frozen = frozen + #{quantity} " +
            "WHERE id = #{productId} AND (stock - frozen) >= #{quantity}")
    int freezeStock(@Param("productId") Long productId, @Param("quantity") Integer quantity);

    @Update("UPDATE product SET stock = stock - #{quantity}, frozen = frozen - #{quantity} " +
            "WHERE id = #{productId} AND frozen >= #{quantity}")
    int confirmStock(@Param("productId") Long productId, @Param("quantity") Integer quantity);

    @Update("UPDATE product SET frozen = frozen - #{quantity} " +
            "WHERE id = #{productId} AND frozen >= #{quantity}")
    int cancelStock(@Param("productId") Long productId, @Param("quantity") Integer quantity);
}
```

幂等设计：多次操作结果一样

基于Seata实现TCC事务

（3）订单服务

实现三个接口

```java
@Slf4j
@Service
public class OrderTccServiceImpl implements OrderTccService {
    @Autowired
    private OrderMapper orderMapper;

    @Override
    @Transactional
    public boolean tryCreateOrder(Long userId, Long productId, Integer quantity, BusinessActionContext context) {
        log.info("Order Try: 创建待确认订单, userId={}, productId={}, quantity={}", userId, productId, quantity);
        String txId = context.getXid();
        // 幂等检查
        if (orderMapper.existsByTxId(txId) > 0) {
            log.warn("订单已存在，幂等返回成功");
            return true;
        }
        Order order = new Order();
        order.setUserId(userId);
        order.setProductId(productId);
        order.setQuantity(quantity);
        order.setStatus(0); //状态设置为“未确认”
        order.setTxId(txId);
        int rows = orderMapper.insert(order);
        if (rows == 1) {
            context.getActionContext().put("txId", txId);
            return true;
        }
        return false;
    }
```

基于Seata实现TCC事务

（3）订单服务实现三个接口

```java
@Override
@Transactional
public boolean confirm(BusinessActionContext context) {
    Map<String, Object> params = context.getActionContext();
    String txId = (String) params.get("txId");
    log.info("Order Confirm: 确认订单, txId={}", txId);
    int rows = orderMapper.confirmOrder(txId);
    return rows == 1;
}

@Override
@Transactional
public boolean cancel(BusinessActionContext context)
    Map<String, Object> params = context.getActionCon
    String txId = (String) params.get("txId");
    log.info("Order Cancel: 取消订单, txId={}", txId);
    int rows = orderMapper.cancelOrder(txId);
    return true;
}
```

```java
@Mapper
public interface OrderMapper {
    @Insert("INSERT INTO `order` (user_id, product_id, quantity, status, tx_id) "
            "VALUES (#{userId}, #{productId}, #{quantity}, #{status}, #{txId})")
    @Options(useGeneratedKeys = true, keyProperty = "id")
    int insert(Order order);

    @Select("SELECT COUNT(*) FROM `order` WHERE tx_id = #{txId}")
    int existsByTxId(String txId);

    @Update("UPDATE `order` SET status = 1 WHERE tx_id = #{txId} AND status = 0")
    int confirmOrder(String txId);

    @Update("UPDATE `order` SET status = 2 WHERE tx_id = #{txId} AND status = 0")
    int cancelOrder(String txId);
}
```

幂等设计：多次操作结果一样

基于Seata实现TCC事务

（4）下单服务启用全局事务

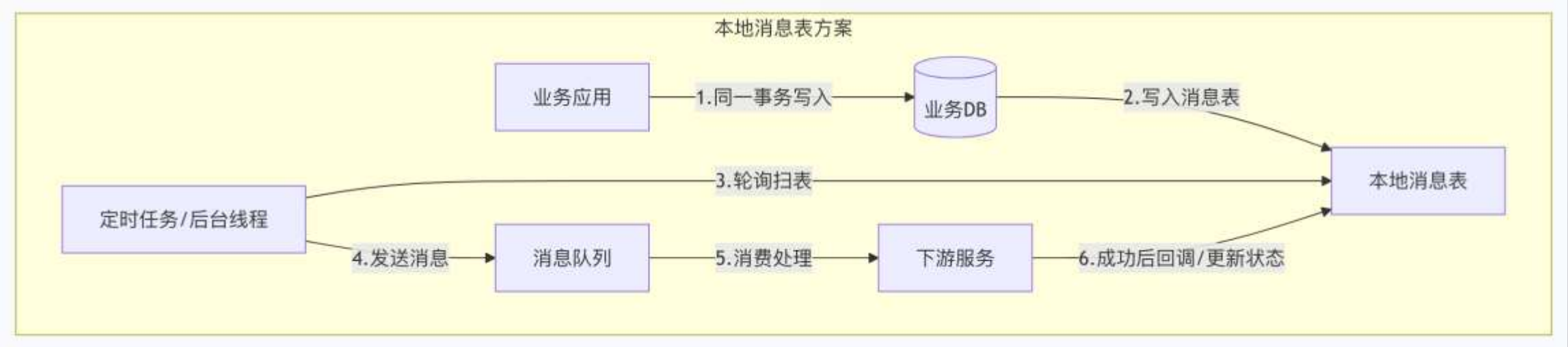
```java
@Slf4j
@Service
public class OrderService {

    @Autowired
    private ProductTccService productTccService;
    @Autowired
    private OrderTccService orderTccService;

    @GlobalTransactional(name = "create-order-tcc", rollbackFor = Exception.class)
    public void createOrder(Long userId, Long productId, Integer quantity) {
        log.info("开始 TCC 全局事务: 下单扣库存");
        // 1. 库存服务 Try
        boolean stockResult = productTccService.tryDeductStock(productId, quantity, null);
        if (!stockResult) {
            throw new RuntimeException("库存不足, 下单失败");
        }
        // 2. 订单服务 Try
        boolean orderResult = orderTccService.tryCreateOrder(userId, productId, quantity, null);
        if (!orderResult) {
            throw new RuntimeException("创建订单失败");
        }
        // 若所有 Try 成功, Seata 自动调用各参与方的 Confirm
        log.info("TCC 事务执行完成");
    }
}
```

TCC事务优化

- **TCC事务可以优化，支持高并发**
  - **Redis 预过滤：**拦截无效请求，只让通过 Redis 校验的请求进入 TCC。
  - **Try 阶段轻量化：**仅做必要的资源预留（如更新 frozen 字段），立即提交本地事务。
  - **Confirm/Cancel 异步化：**将 Confirm/Cancel 改为异步执行，通过消息队列触发。
  - **批量 Try 与批量 Confirm：**将多个 Try 操作合并为一次 RPC 调用，减少网络往返。
  - **热点数据分片：**将热点数据（如库存）拆分成多个分片，来分散压力。

基于消息的最终一致性

- **场景**
  - 极高并发场景，不要求强一致，只保障最终一致。
- **思路（eBay提出的方案）**
  - 把跨多个服务的复杂事务，拆解成多个在单个数据库内就能完成的简单本地事务，然后通过消息和重试机制把它们串联起来。
- **方案一：本地消息表**
  - 使用本地消息表持久化，**避免消息丢失，支持重试**。



本地消息表方案的实现

- 创建订单，并写入消息表

```sql
CREATE TABLE `message_outbox` (
    `id` BIGINT NOT NULL AUTO_INCREMENT,
    `message_id` VARCHAR(64) NOT NULL COMMENT '全局唯一消息ID，用于幂等
    `topic` VARCHAR(128) NOT NULL COMMENT 'Routing Key 或 Exchange 名
    `payload` TEXT NOT NULL COMMENT '消息体(JSON格式)',
    `status` VARCHAR(20) NOT NULL DEFAULT 'PENDING' COMMENT 'PENDING
    `retry_count` INT DEFAULT 0,
    `max_retry` INT DEFAULT 5,
    `next_retry_time` DATETIME DEFAULT CURRENT_TIMESTAMP,
    `created_at` DATETIME DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (`id`),
    UNIQUE KEY `uk_message_id` (`message_id`),
    KEY `idx_status_next_retry` (`status`, `next_retry_time`)
) ENGINE=InnoDB;
```

问：为什么要写入消息表，而不是直接在方法中发送消息？

```java
/**
 * 创建订单（业务操作 + 写本地消息表）
 */
@Transactional(rollbackFor = Exception.class)
public void createOrder(OrderRequest request) {
    // 1. 执行本地业务：创建订单
    Order order = new Order();
    order.setUserId(request.getUserId());
    order.setProductId(request.getProductId());
    order.setStatus("CREATED");
    orderMapper.insert(order);

    // 2. 构建扣减库存的消息
    InventoryDeductMsg msg = new InventoryDeductMsg();
    msg.setOrderId(order.getId());
    msg.setProductId(request.getProductId());
    msg.setQuantity(request.getQuantity());

    // 3. 写入本地消息表（与业务操作在同一事务中）
    MessageOutbox outbox = new MessageOutbox();
    outbox.setMessageId(IdGenerator.nextId());    // 全局唯一ID
    outbox.setTopic("inventory.deduct");
    outbox.setPayload(JSON.toJSONString(msg));
    outbox.setStatus("PENDING");
    outboxMapper.insert(outbox);

    // 此时事务提交，业务数据和消息记录同时持久化
}
```

本地消息表方案的实现

- 订单服务定时发送消息，如果失败进行重试

```java
/**
 * 定时扫描待发送消息，发送到 Kafka
 * 注意：Kafka 不支持原生延迟消息，需要重试时可借助延迟队列组件
 */
@Scheduled(fixedDelay = 5000)  // 每5秒扫描一次
public void sendPendingMessages() {
    // 只发送5分钟前的消息，避免与刚创建的PENDING消息冲突
    LocalDateTime maxAge = LocalDateTime.now().minusMinutes(5);
    List<MessageOutbox> pendingMessages = outboxRepository.findPendingMessages(maxAge);

    for (MessageOutbox message : pendingMessages) {
        try {
            // 发送到 Kafka
            CompletableFuture<SendResult<String, String>> future =
                kafkaTemplate.send(message.getTopic(), message.getMessageId(), message.getPayload());
            // 同步等待发送结果
            future.get(10, TimeUnit.SECONDS);
            // 发送成功，更新状态
            outboxRepository.markAsSent(message.getMessageId());
            log.info("消息发送成功, messageId: {}", message.getMessageId());
        } catch (Exception e) {
            log.error("消息发送失败, messageId: {}, 重试次数: {}",
                message.getMessageId(), message.getRetryCount(), e);
            // 更新重试次数
            message.setRetryCount(message.getRetryCount() + 1);
            outboxRepository.save(message);
        }
    }
}
```

本地消息表方案的实现

- 库存服务监听消息扣除库存

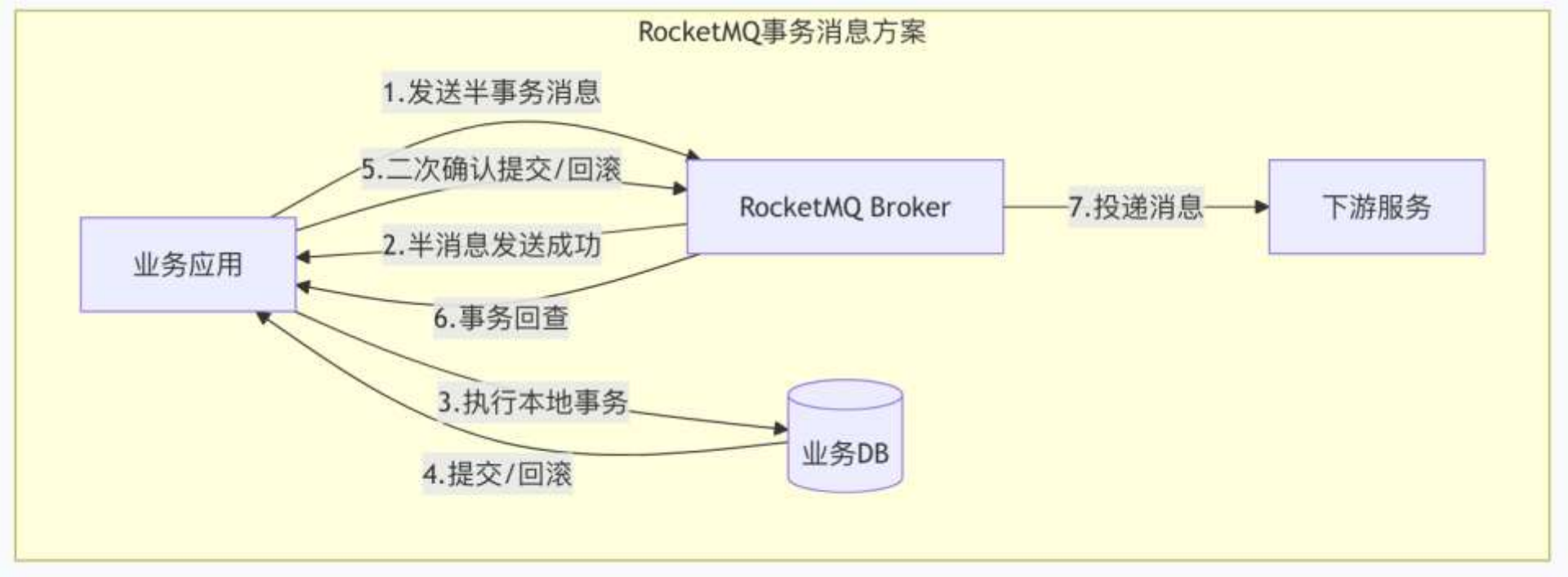
```java
@Service
@Slf4j
public class InventoryService {
    @Autowired
    private ProductMapper productMapper;

    @Transactional(rollbackFor = Exception.class)
    public void deductStock(Long productId, Integer quantity) {
        // 使用乐观锁更新库存
        int rows = productMapper.deductStockWithVersion(productId, 
        if (rows == 0) {
            throw new RuntimeException("库存不足或乐观锁冲突");
        }
        log.info("库存扣减成功, productId: {}, quantity: {}", productI
    }
}
```

```java
/**
 * 消费库存扣减消息
 *
 * @param record 消息记录
 * @param ack    Acknowledgment 用于手动提交偏移量
 */
@KafkaListener(topics = "inventory.deduct", groupId = "inventory-group")
public void consume(ConsumerRecord<String, String> record, Acknowledgment ack) {
    String messageId = record.key();
    String payload = record.value();
    log.info("收到消息, messageId: {}, offset: {}", messageId, record.offset());
    // 1. 幂等性检查：基于 Redis 或数据库记录已处理的消息ID
    //    使用 SETNX 保证原子性，设置24小时过期
    Boolean processed = redisTemplate.opsForValue()
        .setIfAbsent("processed_msg:" + messageId, "1", Duration.ofHours(24));
    if (Boolean.FALSE.equals(processed)) {
        log.warn("消息已处理过, 跳过。messageId: {}", messageId);
        ack.acknowledge();  // 确认消费
        return;
    }
    try {
        // 2. 解析消息并执行业务（在自己的本地事务中）
        InventoryDeductMsg msg = JSON.parseObject(payload, InventoryDeductMsg.class);
        inventoryService.deductStock(msg.getProductId(), msg.getQuantity());
        // 3. 业务处理成功，手动提交偏移量
        ack.acknowledge();
        log.info("库存扣减成功, messageId: {}", messageId);
    } catch (Exception e) {
        log.error("库存扣减失败, messageId: {}", messageId, e);
        // 业务失败：删除幂等标记，允许重试
        redisTemplate.delete("processed_msg:" + messageId);
        // 注意：这里不调用 ack.acknowledge()
        // 根据重试策略决定是否抛出异常让 Kafka 重新投递
        throw new RuntimeException("库存扣减失败，需要重试", e);
    }
}
```

RocketMQ 事务消息

方案二：RocketMQ 事务消息

- 思想：利用RocketMQ的事务消息，代替本地消息表和定时发送MQ消息功能。
- 过程：发送半消息（Half Message）执行本地事务。根据本地事务结果，Commit 或 Rollback 半消息。若超时未确认，MQ Server 回查本地事务状态。



武漢大學 WUHAN UNIVERSITY

事务模型的比较

方案	一致性	性能	业务侵入	适用场景
XA (2PC)	强一致	低 (锁库)	无 (数据库支持)	银行核心、资金类，对一致性要求极高且并发不高的场景
TCC	最终一致（强）	高 (锁业务)	高 (需实现Try/Confirm)	交易系统、金融支付、中高并发场景
SAGA	最终一致	中	中 (需定义补偿)	长流程、微服务链路过长、遗留系统集成
消息最终一致	最终一致（弱）	高	低 (MQ)	旁路逻辑（下单发券、写日志）、异步解耦

- 分布式事务没有“银弹”，只有“权衡”。
- 从 2PC 的强一致刚性事务，到 TCC/SAGA 的柔性事务，核心思想是 CAP 下的权衡

4、 高并发下的一致性

场景：库存秒杀系统中的一致性要求

场景	并发性	一致性要求
库存扣减 + 订单创建（核心）	极高并发	一致性要求高：不能超卖（库存扣减必须准确），不能重复创建订单
支付回调 + 订单更新	低并发	最终一致性为主：支付结果必须准确更新，但允许短暂延迟 幂等性至关重要：支付平台可能会重复发送回调，必须保证只处理一次。
订单取消+库存回滚	中等并发	一致性要求高：不能多回滚（库存变多），也不能少回滚
纯异步旁路操作（如日志、统计）	极高并发	对一致性要求低

各种事务方案的并发能力

方案	典型并发能力（参考）	核心限制因素
消息最终一致性	10万+ QPS	消息队列吞吐能力、下游消费速度
TCC（优化后）	1万 ~ 10万 TPS	热点行锁竞争、分片数量
Seata AT（无锁优化）	5千 ~ 3万 TPS	数据库行锁、全局锁竞争
SAGA	3千 ~ 1万 TPS	业务补偿耗时、状态存储
XA（2PC）	1千 ~ 5千 TPS	长锁、同步阻塞、协调者开销

高并发下的一致性保障手段

优化手段	具体做法
Redis 前置过滤	热点数据（如库存）放在 Redis 中，通过Lua脚本原子处理
消息队列削峰	使用消息队列实现异步写库，减少数据库层并发写压力
热点数据拆分	多个子库存（如 1000 库存拆成 10 个槽位），用户请求通过 hash 分散到不同槽位，减少行锁冲突。
幂等与去重优化	使用轻量级幂等（如基于 Redis 的请求ID去重），减少数据库压力。
事务范围	一致性要求高的场景「扣库存+插入订单」：使用事务 一致性要求低的场景「增加积分、发送消息」：采用异步通知，不用事务
事务模式选择	低并发、强一致性：2PC/3PC事务 中等并发、高一致性：TCC事务 高并发、高一致性：基于消息的最终一致性
数据库隔离级别	将数据隔离级别降低为READ-COMMITTED，避免间隙锁，减少锁冲突
数据库锁类型	采用乐观锁，降低高并发写冲突
数据库索引	确保事务中 where 条件有索引

Redis 前置过滤

- 在Redis中完成库存预扣减，实际数据库扣减可以异步完成。
- 超卖问题：当多个请求同时通过stock > 0检查后，每个请求都会执行减库存操作，导致库存变为负数。

```
// 典型超卖代码示例
@GetMapping("/reduce")
public String reduce() {
    int stock = (Integer) redisTemplate.opsForValue().get("stock");
    if (stock > 0) {
        redisTemplate.opsForValue().set("stock", stock - 1);
        return "秒杀成功";
    }
    return "库存不足";
}
```

Redis 前置过滤

- Lua原子扣减脚本：防超卖、限购

```java
/**
 * 执行秒杀预扣减
 * @param productId 商品ID
 * @param userId    用户ID
 * @param quantity  购买数量
 * @param maxBuy    限购数量
 * @return 秒杀结果状态码：0-库存不足，1-超过限购，2-成功
 */
public int doSeckill(Long productId, String userId, int quantity, int maxBuy) {
    String stockKey = "seckill:stock:" + productId;
    String userKey = "seckill:user:" + productId;

    List<String> keys = Arrays.asList(stockKey, userKey);
    Long result = redisTemplate.execute(
        seckillScript,
        keys,
        String.valueOf(quantity),
        userId,
        String.valueOf(maxBuy)
    );
    return result.intValue();
}
```

```lua
-- KEYS[1] : 库存key
-- KEYS[2] : 用户购买记录key (hash)
-- ARGV[1] : 商品数量 (默认1)
-- ARGV[2] : 用户ID
-- ARGV[3] : 限购数量
-- 返回值: 0=库存不足, 1=超限, 2=成功, 3=其他错误

local stock_key = KEYS[1]
local user_key = KEYS[2]
local quantity = tonumber(ARGV[1])
local user_id = ARGV[2]
local max_buy = tonumber(ARGV[3])

-- 1. 检查库存
local stock = redis.call('GET', stock_key)
if not stock or tonumber(stock) < quantity then
    return 0   -- 库存不足
end

-- 2. 检查用户限购
local bought = redis.call('HGET', user_key, user_id)
bought = tonumber(bought) or 0
if bought + quantity > max_buy then
    return 1   -- 超过限购
end

-- 3. 扣减库存
redis.call('DECRBY', stock_key, quantity)
-- 4. 增加用户购买记录
redis.call('HINCRBY', user_key, user_id, quantity)
-- 可选: 设置过期时间 (活动结束后清理)
redis.call('EXPIRE', user_key, 86400)

return 2   -- 成功
```

乐观锁：高并发下数据库访问

乐观锁与悲观锁

- 并发场景下，数据库执行写操作时，特别是先读后写时，为避免其他线程修改数据，要对写入的行加锁，来保障数据一致性和事务隔离。

维度	悲观锁	乐观锁
核心思想	假设冲突必然发生，在操作数据前先加锁，阻塞其他事务	假设冲突很少发生，不加锁，提交时检查是否被修改
处理时机	读取数据时即锁定，直到事务结束	只在提交时检查冲突，若有则回滚重试
并发性能	低，锁竞争导致吞吐量下降	高，无锁机制适合读多写少场景
实现方式	SELECT ... FOR UPDATE	版本号（version）、时间戳、CAS等

悲观锁的实现

```sql
BEGIN;
-- 对行加排他锁
SELECT * FROM product WHERE id = 1 FOR UPDATE;
-- 业务逻辑，例如检查库存后扣减
UPDATE product SET stock = stock - 1 WHERE id = 1;
COMMIT;
```

```java
@Transactional
public void deductStock(Long productId, int quantity) {
    // 1. 悲观锁查询，锁定该行
    Product product = productMapper.selectForUpdate(productId);
    // 2. 业务校验（例如库存是否充足）
    if (product.getStock() >= quantity) {
        // 3. 执行更新
        productMapper.updateStock(productId, product.getStock() - quantity);
    } else {
        throw new InsufficientStockException();
    }
    // 4. 事务提交后，锁自动释放
}
```

乐观锁的实现

- 实现方法
  - 表中添加version字段
- 乐观锁特点
  - **无锁操作**：不加任何数据库锁。
  - **重试机制**：业务层需处理更新失败，通常循环重试若干次。
  - **适用场景**：读多写少、并发冲突较低的场景（如商品信息浏览、用户资料修改）。
- 优缺点
  - 优点：并发性能好，避免死锁
  - 缺点：冲突时重试会增加开销

```
-- 表结构
CREATE TABLE product (
    id INT PRIMARY KEY,
    stock INT,
    version INT
);

-- 读取数据时获取版本号
SELECT id, name, stock, version FROM product WHERE id = #{id}

-- 更新时检查版本号。若影响行数为0，说明期间版本号和读时不同，需重试（重新查询并再次尝试）
UPDATE product SET stock = stock - #{quantity}, version = version + 1
WHERE id = #{id}  AND version = #{version}
```

乐观锁的实现

```java
@Mapper
public interface ProductMapper {

    @Select("SELECT id, name, stock, version FROM product WHERE id = #{id}")
    Product selectById(Long id);

    @Update("UPDATE product SET stock = stock - #{quantity}, version = version + 1 " +
            "WHERE id = #{id} AND stock >= #{quantity} AND version = #{version}")
    int deductStockWithVersion(@Param("id") Long id,
                               @Param("quantity") Integer quantity,
                               @Param("version") Integer version);
}
```

```java
/**
 * 扣减库存（带重试）
 * @param productId 商品ID
 * @param quantity  扣减数量
 * @return 是否成功
 */
@Transactional
public boolean deductStockWithRetry(Long productId,
    int maxRetry = 3; // 重试3次
    for (int i = 0; i < maxRetry; i++) {
        // 1. 查询商品，获取当前版本号
        Product product = productMapper.selectById(productId);
        if (product == null || product.getStock() < quantity) {
            return false; // 库存不足
        }
        Integer version = product.getVersion();
        // 2. 乐观锁更新
        int rows = productMapper.deductStockWithVersion(productId, quantity, version);
        if (rows == 1) {
            return true; // 更新成功
        }
    }
    return false; // 重试多次仍失败
}
```

武漢大學 WUHAN UNIVERSITY

作业：分布式事务

事务与一致性

- 场景：在商品秒杀系统中，订单服务和库存服务是两个独立的微服务，分别有各自数据库
- 1、在秒杀下单时，基于Redis实现库存预扣减，防超卖、限购
- 2、采用基于消息的一致性或TCC事务保障数据一致性
  - 下单+库存扣减一致性
  - 订单支付+ 订单状态更新一致性